BRTGC Engineering Assessment

Project Title

HRFlow — HR & Internal Communication Management Platform

Objective

Build a modern, secure, and scalable **HR and Internal Communication Management Platform** for an organization.

HRFlow should provide organizations with a centralized platform to:

- Manage employees
- Manage hierarchical departments
- Track employee attendance
- Manage employee leave requests
- Send internal communications
- Share files and documents through messages
- Monitor important HR and organizational activities

The purpose of this assessment is to evaluate your ability to design and implement a real-world business application while demonstrating:

- Engineering thinking
- Software architecture
- Database modelling
- Problem-solving ability
- Product thinking
- UI/UX implementation
- Code maintainability
- Type safety
- Security awareness
- Ability to build scalable business systems

You may use AI tools during development. However, you are expected to **fully understand the implementation**, including the architecture, database design, business rules, and technical decisions made during development.

Required Technology Stack

You must use the following technologies:

- **Next.js** — App Router
- **TypeScript**
- **Prisma ORM**
- **MongoDB**

You may additionally use:

- Tailwind CSS
- Zustand
- Redux
- Context API
- React Query
- SWR
- NextAuth
- Custom authentication
- Socket.io
- Pusher
- Any suitable UI component library

The choice of additional libraries should be justified based on the requirements of the application.

Core Features

1. Authentication & Authorization

Implement a secure authentication and authorization system.

Authentication

Users should be able to:

- Register
- Log in
- Log out
- Maintain a persistent session
- Access protected application areas

The implementation should include:

- Secure password hashing

- Input validation
- Authentication error handling
- Session persistence
- Protected routes
- Secure session management

User Roles

The system should support at least the following roles:

- **Admin**
- **HR Manager**
- **Employee**

Roles should determine what actions and resources a user can access.

Examples:

- Admin can manage the organization and its users.
- HR Managers can manage employees, departments, attendance and leave workflows according to their permissions.
- Employees can access their own information, attendance, leave requests and authorized communications.

Authorization should be enforced **server-side**, not only through UI visibility.

2. Employee Management

The system should provide employee management functionality.

Each employee should contain, at minimum:

- Full name
- Email address
- Phone number
- Department
- Position
- Employment status
- Date joined
- Profile image — optional

Employee Operations

Authorized users should be able to:

- Create employees
- Edit employee information
- View employee profiles
- Search employees
- Filter employees
- View employees by department
- Deactivate employees

Employee deactivation should preferably preserve historical records such as:

- Attendance
- Leave requests
- Messages
- Activities

Deleting an employee should therefore not unnecessarily destroy historical business records.

3. Department Management

Departments should be modeled as a **hierarchical organizational structure**, rather than as a flat list.

Each department should contain:

- Department name
- Description
- Department head
- Parent department — optional
- Child departments

Example Department Structure

Organization

|

├— Human Resources

| ├— Recruitment

| ├— Training & Development

| └— Employee Welfare

|

├— Operations

| ├── Field Operations

| └── Maintenance

|

└── Finance

├── Accounts

└── Audit

A department can therefore have:

- A parent department
- Multiple child departments
- A department head
- Multiple employees

Department Operations

Authorized users should be able to:

- Create departments
- Edit departments
- View department details
- Assign department heads
- Assign employees to departments
- Move departments within the hierarchy
- View child departments
- Navigate to parent departments
- View employees within a department

The system should maintain both **forward and backward relationships** within the department hierarchy.

For example:

Recruitment

↓

Human Resources

↓

Organization

Recruitment should be able to identify its parent departments, while Human Resources should be able to identify its child departments.

Hierarchy Validation

The implementation should prevent invalid organizational structures.

Examples:

- A department cannot be its own parent.
- A department cannot become a descendant of itself.
- Circular department relationships must be prevented.
- A department head should be an active employee.
- Appropriate authorization should be required to modify the organizational hierarchy.

4. Attendance Management

Employees should be able to manage their daily attendance.

Employee Actions

Employees should be able to:

- Clock in
- Clock out
- View their attendance history
- View their attendance for a selected period

Attendance Requirements

The system should maintain daily attendance records.

The implementation should:

- Prevent duplicate clock-ins for the same attendance period
- Prevent invalid clock-out operations
- Validate attendance state transitions
- Record timestamps
- Preserve attendance history

Attendance Statistics

The dashboard should be able to display statistics such as:

- Employees present today
- Employees absent
- Employees who have not clocked out
- Attendance history

- Attendance trends

5. Leave Request System

Employees should be able to submit leave requests through the platform.

Leave Request Fields

Each leave request should contain:

- Leave type
- Start date
- End date
- Reason
- Status
- Requester
- Submission date
- Review information where applicable

Employee Actions

Employees should be able to:

- Submit leave requests
- View their leave requests
- View leave status
- Review previous leave requests

HR/Admin Actions

Authorized HR Managers and Admins should be able to:

- View pending leave requests
- Approve leave requests
- Reject leave requests
- Review leave history

Leave Status

At minimum, support:

PENDING

APPROVED

REJECTED

The system should validate:

- Start date
- End date
- Date ordering
- Invalid or overlapping requests where applicable
- Unauthorized approval/rejection
- Invalid status transitions

The leave workflow should preserve an appropriate audit trail of important actions.

6. Internal Messaging & Organizational Communication

HRFlow should provide an internal communication system that supports communication between employees and departments.

The messaging system should support:

1. Employee-to-Employee messaging
2. Employee-to-Department messaging
3. Department-to-Department messaging
4. Department-to-Employee messaging

The system should use a reusable conversation/message architecture rather than implementing each communication type as a completely separate system.

6.1 Employee-to-Employee Messaging

Employees should be able to:

- Send private messages to another employee
- View conversation history
- Reply to messages
- View message timestamps
- View sender information
- Attach files to messages

Privacy

Employee-to-employee conversations are considered **individual/private communications**.

Department hierarchy should **not automatically grant department heads access to private employee conversations**.

For example:

HR

│

└── Recruitment

  │

  ├── Alice

  └── Bob

If Alice sends a private message to Bob:

Alice ————————> Bob

The HR Head should not automatically gain access to the conversation simply because Alice and Bob belong to departments below HR.

6.2 Employee-to-Department Messaging

Employees should be able to send communications to an entire department.

Example:

Employee

  │

  └————————> HR Department

A message may contain:

- Text
- Attachments
- Timestamp
- Sender information
- Department recipient

Department members and authorized department users should be able to access the communication according to the organization's communication permissions.

6.3 Department-to-Department Messaging

Departments should be able to communicate with other departments.

Example:

Human Resources

  │

└──────> Operations

Examples of departmental communications may include:

- Requests
- Announcements
- Reports
- Operational updates
- Administrative communications
- Document sharing

The system should maintain a complete conversation history.

6.4 Department-to-Employee Messaging

Authorized department users should be able to send messages to individual employees.

Example:

HR Department

│

└──────> Employee

This can be used for:

- HR notifications
- Requests
- Administrative communication
- Employee-specific instructions
- Document sharing

6.5 Department Hierarchy & Message Visibility

Departmental communication should take the organization's department hierarchy into account.

For example:

Organization

│

├── Human Resources

│ └── Recruitment

│

└── Operations

If the **Recruitment Department** sends a departmental communication, the communication may be visible to authorized department heads within its organizational hierarchy.

The hierarchy can therefore be resolved upward:

Recruitment

↑

Human Resources

↑

Organization

This means an appropriate departmental communication originating from Recruitment can be accessible to:

- Recruitment Head
- Human Resources Head
- Higher-level authorized department heads

subject to the application's authorization rules.

However, this visibility rule applies to **departmental communications only**.

Individual Communication Remains Private

The hierarchy must not automatically expose private employee-to-employee conversations.

Therefore:

Employee ↔ Employee

is private unless the application's explicit authorization model provides otherwise.

While:

Department ↔ Department

Employee → Department

Department → Employee

can participate in organizational communication visibility.

The implementation should clearly distinguish between **individual communication** and **departmental communication**.

6.6 Messaging Data Model

The messaging architecture should be designed around reusable concepts such as:

Conversation

```
   |
   ├── Participants
   |
   └── Messages
         |
         └── Attachments
```

A conversation may involve:

- Employees
- Departments

Messages should contain information such as:

- Sender
- Conversation
- Content
- Timestamp
- Attachments
- Message status where applicable

The database design should support future expansion without requiring a complete rewrite of the messaging system.

6.7 Realtime Messaging

Realtime messaging is **optional but highly valued**.

Candidates may implement realtime communication using technologies such as:

- Socket.io
- Pusher
- Other appropriate realtime infrastructure

A basic implementation without realtime functionality is acceptable, provided the messaging architecture is well designed.

7. File Sharing Through Messaging

File sharing should be implemented as an **integrated capability of the messaging system**.

Users should be able to attach files directly to messages.

Example

Message

│

├── Text

│

├── PDF attachment

│

└── Image attachment

Supported file examples include:

- PDF files
- Images
- Word/document files
- Other appropriate business documents

Expected Functionality

Users should be able to:

- Upload a file while composing a message
- Attach files to messages
- View attachment information
- Preview supported files where possible
- Download attachments
- Access attachments through the relevant conversation

File Metadata

The database should store metadata such as:

- Original file name
- MIME type
- File size
- Storage reference/key
- Upload timestamp
- Message relationship

The actual file content should preferably be stored using appropriate file/object storage rather than directly inside MongoDB.

The database should contain the metadata and secure reference required to access the file.

Security

File uploads should include appropriate:

- File type validation
- File size validation
- Access control
- Secure file references
- Authorization checks

A user should not be able to access an attachment merely by guessing its URL.

Optional Enhancements

Candidates may implement:

- PDF previews
- Image previews
- Document previews where supported
- Drag-and-drop uploads
- Upload progress
- Multiple attachments
- Attachment thumbnails

8. Dashboard

Create a responsive dashboard providing an overview of the organization's current state.

The dashboard should display information such as:

- Total employees
- Employees present today
- Pending leave requests
- Total departments
- Recent activities
- Recent messages

Dashboard information should respect the user's role and permissions.

For example, an employee should not necessarily receive the same organization-wide statistics available to an Admin or HR Manager.

Dashboard UX

The dashboard should provide:

- Responsive layout
- Mobile-friendly interface
- Clear information hierarchy
- Loading states
- Empty states
- Error states
- Appropriate visual feedback
- Easy navigation to frequently used features

Bonus Features

The following features are optional but highly valued.

Realtime Messaging

Implement realtime message delivery using technologies such as Socket.io or Pusher.

Notifications

Implement notifications for events such as:

- New messages
- Leave request submissions
- Leave approvals
- Leave rejections
- Department communications
- Other important organizational activities

Email Integration

Send emails for selected events, such as:

- Account-related notifications
- Leave updates
- Important organizational communications

Dark Mode

Provide a responsive light/dark theme experience.

PWA Support

Implement Progressive Web App functionality where appropriate.

Offline Support

Provide limited offline capabilities where appropriate.

Activity/Audit Logs

Track important system activities, such as:

- Employee creation
- Employee updates
- Employee deactivation
- Department changes
- Leave approvals/rejections
- Attendance actions
- Important messaging activities

Analytics

Provide charts or visual analytics for relevant HR information such as:

- Attendance trends
- Leave trends
- Employee distribution
- Department statistics

File Previews

Allow users to preview supported attachments directly within conversations.

Message Reactions

Allow users to react to messages using predefined reactions.

Docker Support

Provide Docker configuration for local development or deployment.

API Documentation

Provide documentation for important application APIs.

Important Engineering Expectations

We are **not only checking whether the application works**.

We are evaluating how you think about building a production-quality business system.

Your implementation will be assessed based on:

Architecture

- Clear project structure
- Separation of concerns
- Reusable business logic
- Maintainable components
- Appropriate server/client boundaries

Database Design

- Appropriate Prisma models
- Correct relationships
- Proper indexing
- Data integrity
- Scalable modelling decisions
- Appropriate MongoDB usage

Particular attention will be given to the modelling of:

- Employees
- Departments
- Department hierarchy
- Attendance
- Leave requests
- Conversations
- Messages
- Message participants
- Message attachments

Type Safety

The application should make effective use of TypeScript.

Avoid unnecessary:

any

and ensure data flowing between the UI, server, validation layer and database is appropriately typed.

Validation

Use proper validation for:

- Authentication
- Employee data

- Department data
- Attendance operations
- Leave requests
- Messages
- File uploads

Authorization

Authorization should be enforced on the server.

Do not rely solely on:

"If the button is hidden, the user cannot perform the action."

The underlying server operation must also verify permissions.

Error Handling

The application should handle:

- Validation errors
- Authentication errors
- Authorization errors
- Database errors
- Upload errors
- Network failures
- Invalid requests

Errors should provide useful feedback without exposing sensitive implementation details.

API Design

APIs/server actions should be:

- Predictable
- Validated
- Secure
- Consistent
- Easy to maintain

State Management

Use state management appropriately.

Do not introduce a global state library simply because it is available.

Explain why a particular approach was selected.

For example:

- Server state should preferably remain on the server where appropriate.
- Local UI state can remain within components.
- Client-side shared state should be introduced only where it provides meaningful value.

Loading & Empty States

The application should account for:

- Loading states
- Empty lists
- Empty conversations
- No search results
- Upload progress
- Errors
- Successful operations

UI/UX Expectations

The application should feel like a modern business application.

The interface should be:

- Responsive
- Mobile-friendly
- Accessible
- Consistent
- Intuitive
- Visually organized

Particular attention should be given to:

- Navigation
- Dashboard layout
- Employee management
- Department hierarchy visualization
- Leave request workflow
- Attendance interactions
- Messaging interface

- File attachments
- Forms
- Feedback states

The application should work well on both desktop and mobile screen sizes.

Suggested Application Structure

Candidates are free to structure the application according to their architectural preferences.

A possible structure could be:

```
src/
|
 ├— app/
|   ├— (auth)/
|   |   ├— login/
|   |   └— register/
|   |
|   └— (dashboard)/
|       ├— dashboard/
|       ├— employees/
|       ├— departments/
|       ├— attendance/
|       ├— leave/
|       └— messages/
|
 ├— modules/
|   ├— auth/
|   ├— employees/
|   ├— departments/
|   ├— attendance/
|   ├— leave/
|   ├— messaging/
```

```
|   |   ├── conversations/
|   |   ├── messages/
|   |   ├── participants/
|   |   ├── attachments/
|   |   └── visibility/
|   └── dashboard/
|
├── components/
|   ├── ui/
|   ├── forms/
|   └── layout/
|
├── lib/
|   ├── auth/
|   ├── db/
|   ├── permissions/
|   ├── storage/
|   └── validation/
|
└── types/
```

This is only a suggested structure. Candidates may use another structure provided that it demonstrates clear separation of responsibilities and maintainability.

Deliverables

Submit the following:

1. GitHub Repository

Provide the GitHub repository containing the complete project source code.

The repository should include:

- Application source code
- Prisma schema
- Database configuration

- Environment variable example
- Seed data where applicable
- Documentation
- Setup instructions

2. README

The README must contain:

Setup Instructions

Explain how to:

- Clone the repository
- Install dependencies
- Configure environment variables
- Configure MongoDB
- Run Prisma commands where applicable
- Seed the database
- Start the development server
- Build the application for production

Environment Variables

Provide an example such as:

DATABASE_URL=

AUTH_SECRET=

NEXT_PUBLIC_APP_URL=

STORAGE_URL=

STORAGE_ACCESS_KEY=

STORAGE_SECRET_KEY=

Only include variables actually required by the implementation.

Architecture Explanation

Explain:

- Application architecture
- Folder structure
- Authentication approach

- Authorization approach
- Database architecture
- Department hierarchy
- Messaging architecture
- File attachment architecture
- State management decisions
- Storage decisions

Assumptions

Document assumptions made during development.

Examples:

- How department heads are determined
- How employee deactivation works
- How attendance records are created
- How leave conflicts are handled
- How departmental message visibility works
- How file access is authorized

Tradeoffs

Document important technical decisions and alternatives considered.

For example:

Decision:

Use server-side authorization instead of relying solely on

client-side role checks.

Reason:

Prevents unauthorized direct API/server-action access.

Review Discussion

Shortlisted candidates may be invited to explain their implementation.

You should be prepared to discuss:

Architecture

- Why did you structure the project this way?

- Why did you separate certain modules?
- How would the architecture evolve as the application grows?

Database Modelling

- Why did you choose your Prisma models?
- How did you model the department hierarchy?
- How are employee relationships represented?
- How are conversations and participants represented?
- How are message attachments represented?
- How would you handle larger datasets?

Authorization

- How are roles enforced?
- How are department permissions enforced?
- How does departmental message visibility work?
- Why are private employee messages treated differently?

State Management

- Why did you choose your state management approach?
- What state belongs on the server?
- What state belongs on the client?
- How would you handle realtime updates?

Libraries

Be prepared to explain:

- Why you selected particular libraries
- What alternatives you considered
- Why the selected libraries were appropriate for the problem

Scalability

You may be asked:

- What would you change if the organization had 100,000 employees?
- How would you scale messaging?
- How would you handle large file uploads?
- How would you improve search?
- How would you implement realtime messaging at scale?

Areas for Improvement

Explain:

- What you would improve with additional development time
- What you intentionally kept simple
- What you would change before production deployment

Submission Requirements

Before submitting, ensure that:

- The application runs successfully
- Authentication works
- Authorization works
- Employees can be managed
- Departments support hierarchy
- Attendance works
- Leave workflow works
- Messaging works
- Individual conversations remain private
- Departmental communications respect organizational hierarchy
- Files can be attached to messages
- File access is secured
- Dashboard displays relevant information
- Validation is implemented
- Errors are handled appropriately
- Loading and empty states are implemented
- The README is complete
- No secrets are committed
- The repository can be set up by another developer using the README

Submission Window

You have **72 hours from the moment the assessment is assigned** to complete and submit the project.

Good luck.

BRTGC Engineering Team